

Alright, so all the problems I'm solving are dynamic programming problems. So I gotta talk about that.

What I wanna point out: Except for A^* , every problem has some sort of table that forms the solution idea. In every problem, one iteration relies on the solutions of previous iterations, which comes as entries in that table. When I implemented the Knapsack, there was a surprise: The solutions from previous iterations came from different locations as those from LCS and Needleman. LCS and Needleman always need their left and upper neighbor, while Knapsack needs the upper neighbor and another entry in the upper row. This is why Knapsack could be solved in a different manner (much simpler) than the other two.

Could it be that every Dynamic Programming Problem can be formulated using such a table? If yes, how and why do algorithms differ in the elements they use from that table?

Eddy: "This is the key difference between dynamic programming and simple recursion: a dynamic programming algorithm memorizes the solutions of optimal subproblems in an organized, tabular form (a dynamic programming matrix), so that each subproblem is solved just once."

A^*

It appears that there is some controversy around the question whether A^* is a dynamic or greedy algorithm. What's the difference though?

Greedy algorithms: For every iteration, the greedy algorithm chooses the best solution for this very moment. There is no usage of previous solutions. I guess the best example is hill-climbing search.

Dynamic Programming: We can build the solution of a complex problem by combining the solutions of smaller subproblems with optimal substructure. That means, using a policy, we can divide our problem into smaller sub-problems, solve those sub-problems independently of each other, and then use their solutions to find the solutions to the big problem.

So the question is: Can we formulate Dijkstra with a table, as we do with all the other problems?

Trying Dijkstra with a table

So I just performed Dijkstra with a table: Both Axes have all the nodes. The entries encode the distance it takes from x to y . Starting at $x = 0$, we fill in the distance to all neighbors. We then mark the zeros at both axes as visited. Then, from every entry in the table that is unmarked, we choose the smallest value to continue the next iteration with.

So a formulation in a table-style is possible. However, there is no structure here. This may lie in the nature of the problem: We do not know the order of the nodes in advance. This is contrary to, say, LCS, where we know the order of the letters in advance and can plan the filling-out of our table accordingly.

Maybe Sniedovich makes a similar point, I haven't read it yet.